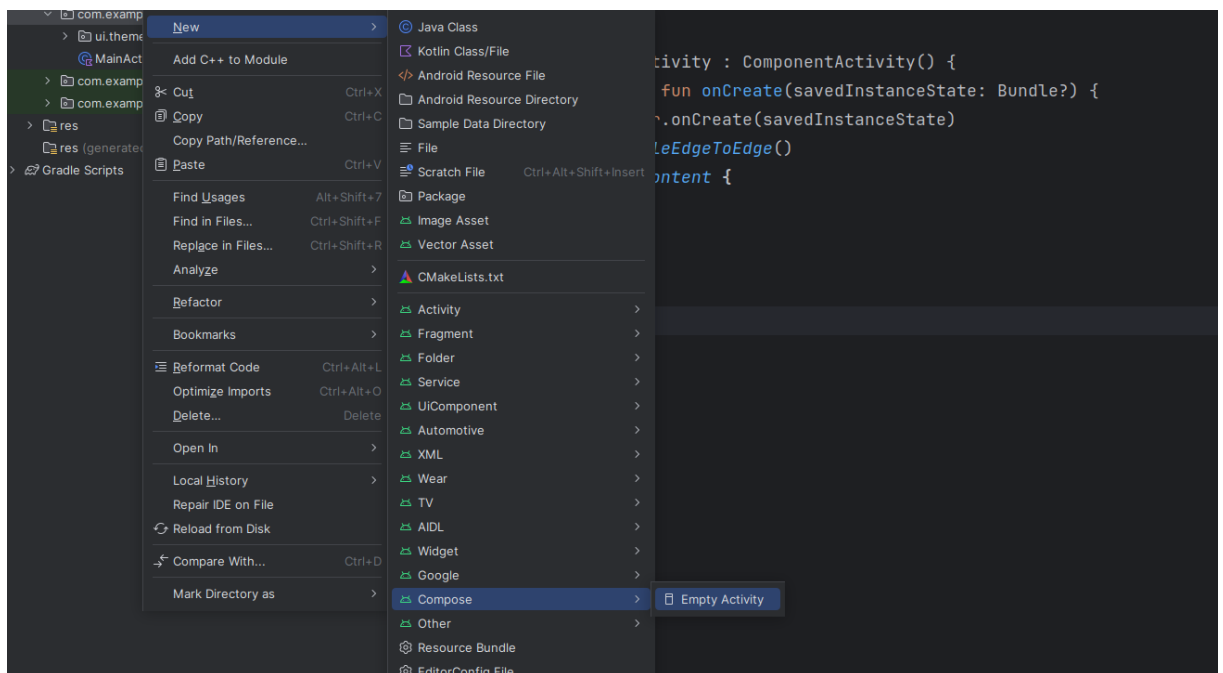


## Android App Development – Data Persistence

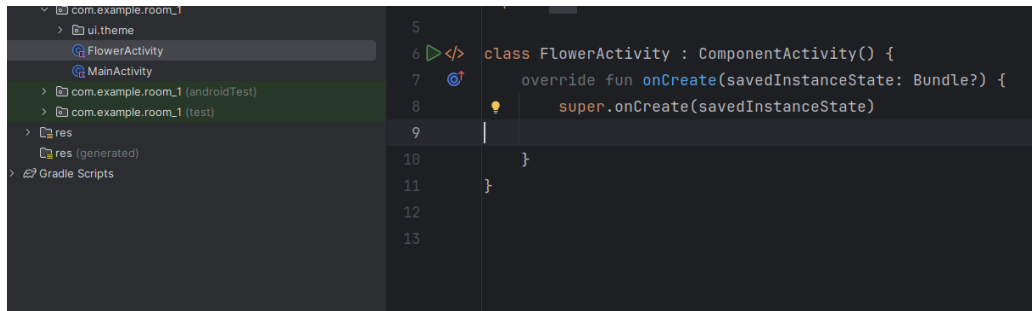
- It is time to learn how to store some data permanently in a local database
- On Android we use [SQLite](#) as the “out-of-the-box” database engine. To make working with DB easier it is recommended to use the [Room](#) library. We will try to create a database of flowers.
- Let’s create a new Empty Activity Project with API 34 as the minimum SDK
- Let’s remove default content and generated functions so the class looks like this:



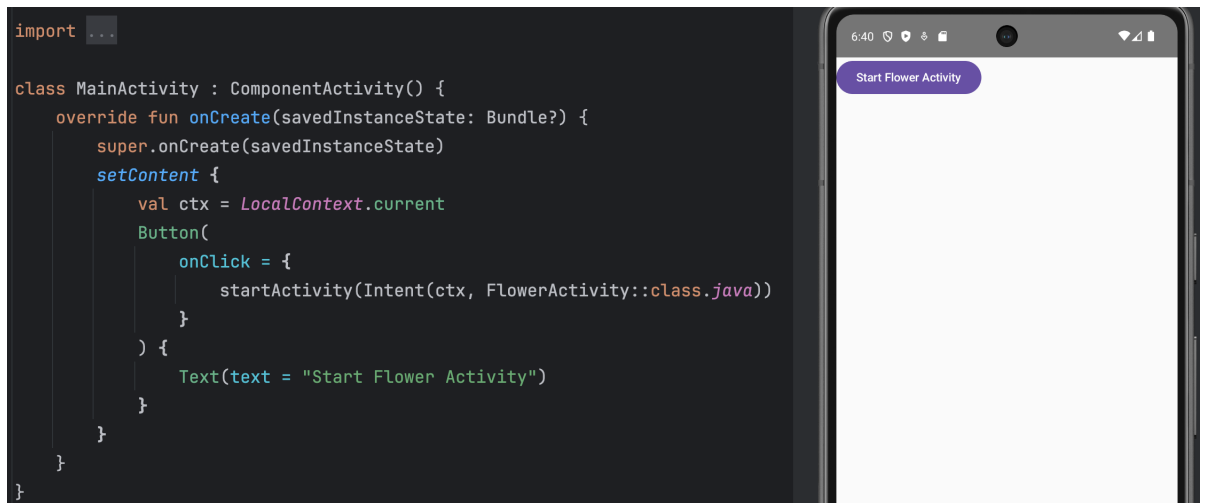
- Let’s add new activity: (New-> Compose -> EmptyActivity)



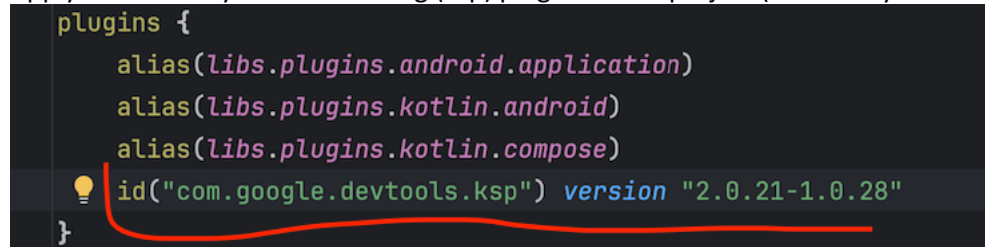
- And let’s call it like FlowerActivity
- Let’s remove set content and generated functions, so the FlowerActivity looks like:



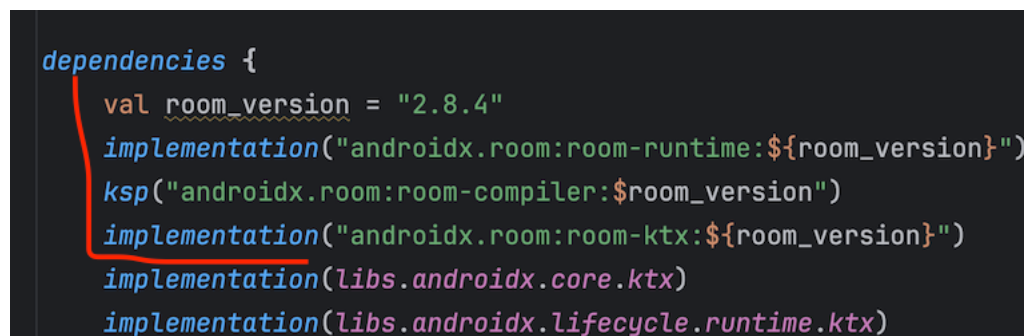
- Let's go back to MainActivity and let's add there a button starting FlowerActivity



- and let's check if it works properly
- To make the Room library available in our project, we need to:
  - Apply the Kotlin Symbol Processing (ksp) plugin for our project (or actually module):

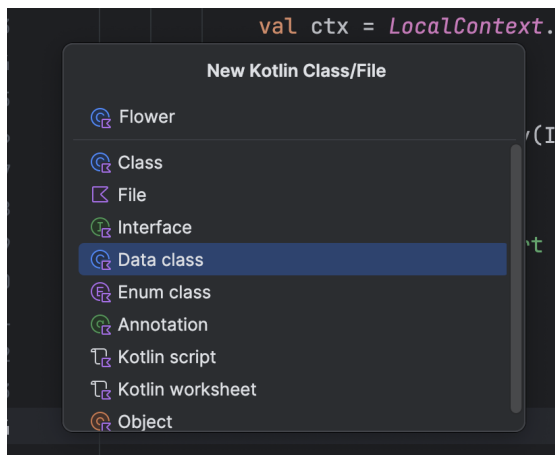


- And, then, we need to add the following dependencies to our build.gradle.kts configuration file:



- And we need to sync our gradle config file.
- Now we can move on to implementing our model.

- First, we have to create the flower entity/data class. So let's add Flower as a new **data** class



- Let's assume, that our app will be a kind of a dictionary comprising the polish and the english name for the flowers covered by the database. So, in our model, each flower has to have two properties: polishName and englishName of String? Type

```
data class Flower(  
    val polishName: String?,  
    val englishName: String?  
)
```

- Next, let's annotate the class with @Entity annotation

```
import androidx.room.Entity  
  
@Entity  
data class Flower(  
    val polishName: String?,  
    val englishName: String?  
)
```

- And let's add the property dedicated to be a primary key:

```
@Entity
data class Flower(
    val flowerID: Int,
    val polishName: String?,
    val englishName: String?
)
```

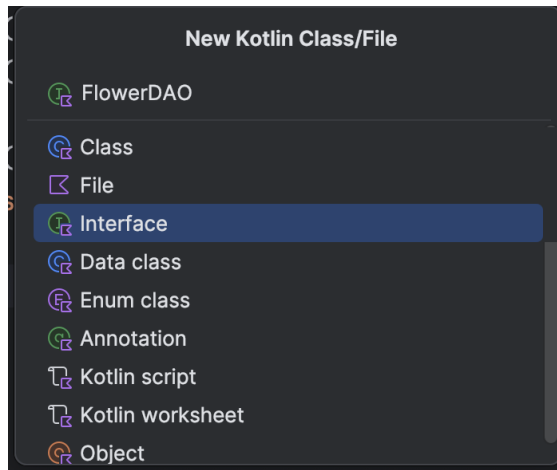
- Next, we need to annotate what is the primary key, and what should be the database column names for our model

```
@Entity
data class Flower(
    @PrimaryKey(autoGenerate = true) val flowerID: Int,
    @ColumnInfo(name = "polish_name") val polishName: String?,
    @ColumnInfo(name = "english_name") val englishName: String?
)
```

- Finally, we need to add the constructor for our Entity class, e.g. as shown below:

```
@Entity
data class Flower(
    @PrimaryKey(autoGenerate = true) val flowerID: Int,
    @ColumnInfo(name = "polish_name") val polishName: String?,
    @ColumnInfo(name = "english_name") val englishName: String?
) {
    constructor(polishName: String?, englishName: String?) :
        this(flowerID: 0, polishName, englishName)
}
```

- flowerID: 0 is just a kind of a placeholder, since we ask room to generate IDs automatically, so it will be replaced accordingly
- The next step is implementing the DAO Interface. So, let's add a new interface called FlowerDao:



- The interface needs to be annotated with @Dao annotation:

```
import androidx.room.Dao

@Dao
interface FlowerDAO {
```

- The interface needs to implement at least the basic CRUD operations (and their “mappings” to SQL statements). You can use for that so-called Convenience methods performing the simple insertions, updates, and deletions without implementing the SQL statement implicitly, e.g. like below:

```
@Dao
interface FlowerDAO {

    @Insert
    fun insertAll(flower: Flower)

    @Delete
    fun delete(flower: Flower)

}
```

- Or you may use so-called “query methods” where you may “map” a function to a given SQL statement like:

```

@Query("SELECT * FROM flower")
fun getAll(): List<Flower>

@Query("SELECT * FROM flower where flowerID IN (:flowerIds)")
fun getAllByIds(flowerIds: IntArray): List<Flower>

```

- So finally, my FlowerDAO implementation looks as follows:
- 

```

import androidx.room.Dao
import androidx.room.Delete
import androidx.room.Insert
import androidx.room.Query

@Dao
interface FlowerDAO {
    @Insert
    fun insertAll(flower: Flower)

    @Delete
    fun delete(flower: Flower)

    @Query("SELECT * FROM flower")
    fun getAll(): List<Flower>

    @Query("SELECT * FROM flower where flowerID IN (:flowerIds)")
    fun getAllByIds(flowerIds: IntArray): List<Flower>
}

```

- Next, we need to add the class representing our database in the project/app. So, let's define the FlowerDatabase class. It needs to be the abstract class extending/inheriting from the RoomDatabase class as shown below:

```

import androidx.room.RoomDatabase

abstract class FlowerDatabase: RoomDatabase() {
}

```

-

- Next we need to annotate the class as a “Database” class, and tell her what entities (which data classes) it should manage. So, in our class it will manage our Flower data class. So let's add the appropriate annotation, eg. as below.

```
@Database(entities = [Flower::class], version = 1)
abstract class FlowerDatabase : RoomDatabase() {
}
```

- When it comes to the version number - we need to bump it up every time when we introduce some changes in our data class(es).
- And next we need to tell the database class that should hold our FlowerDao

```
@Database(entities = [Flower::class], version = 1)
abstract class FlowerDatabase : RoomDatabase() {
    abstract fun flowerDAO(): FlowerDAO
}
```

- And finally, we may work on the application logic using our database.
- So, first, let's build / instantiate our database in the FlowerActivity class:

```
class FlowerActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        val db = buildFlowerDatabase()
    }

    private fun buildFlowerDatabase(): FlowerDatabase {
        val db = Room.databaseBuilder(
            applicationContext,
            FlowerDatabase::class.java, name: "flowerdatabase"
        )
        .allowMainThreadQueries()
        .build()
        return db
    }
}
```

- Next, let's get our flower data access object:

```

class FlowerActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        val db = buildFlowerDatabase()
        val flowerDAO = getFlowerDAO(db)
    }

    private fun getFlowerDAO(db: FlowerDatabase): FlowerDAO {
        val flowerDAO = db.flowerDAO()
        return flowerDAO
    }
}

```

- 
- Then, let's add a sample flower to our database:
- 
- And after running the app, and launching our FlowerActivity, we should be able to see some flowers in our database.

```

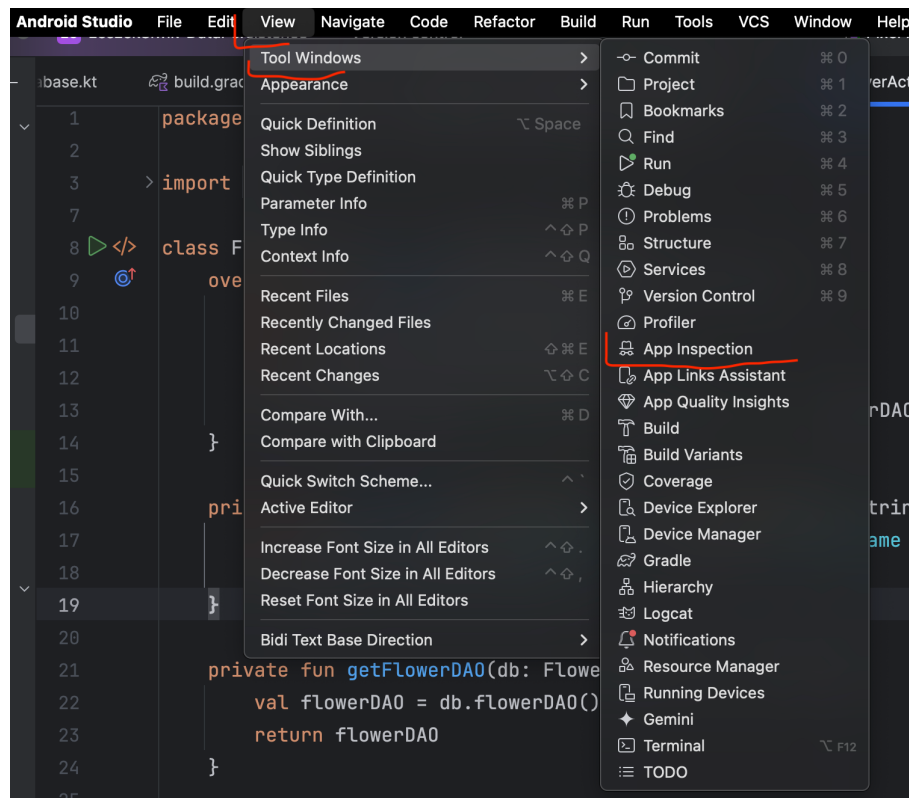
class FlowerActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        val db = buildFlowerDatabase()
        val flowerDAO = getFlowerDAO(db)
        addFlower(polishName: "Roza", englishName: "Rose", flowerDAO)
    }

    private fun addFlower(polishName: String?, englishName: String?, flowerDAO: FlowerDAO) {
        val flower = Flower(polishName = polishName, englishName = englishName)
        flowerDAO.insertAll(flower)
    }
}

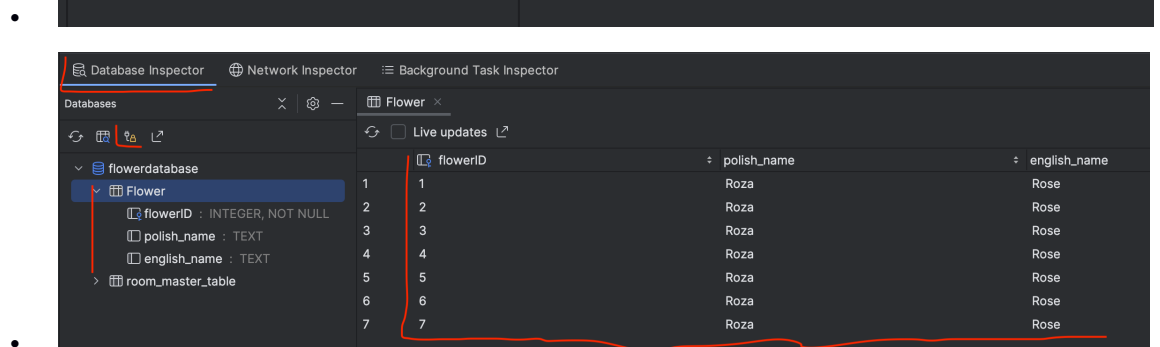
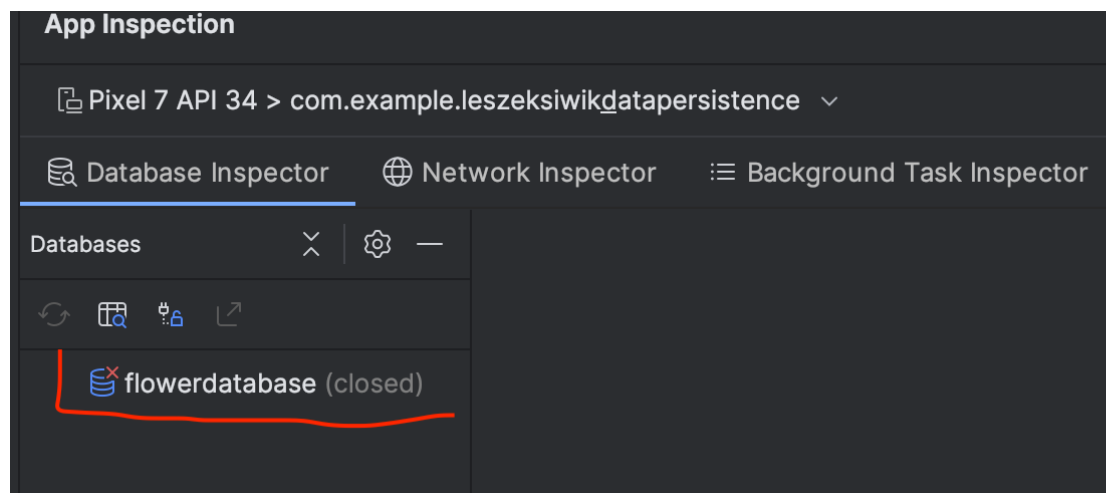
```

- So, let's launch the App Inspector





- 
- Let's connect to our flower database:



- 
- Just, in case, if you encounter any issues, you may compare or just use the implementation of the FlowerActivity that I have at the current stage:

- ```

class FlowerActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        val db = buildFlowerDatabase()
        val flowerDAO = getFlowerDAO(db)
        addFlower("Roza", "Rose", flowerDAO)
    }

    private fun addFlower(polishName: String?, englishName: String?,
flowerDAO: FlowerDAO) {
        val flower = Flower(polishName, englishName)
        flowerDAO.insertAll(flower)
    }

    fun getFlowerDAO(db: FlowerDatabase): FlowerDAO {
        val flowerDAO = db.flowerDAO()
        return flowerDAO
    }

    private fun buildFlowerDatabase(): FlowerDatabase {
        val db = Room.databaseBuilder(
            applicationContext, FlowerDatabase::class.java, "flowers"
        )
            .allowMainThreadQueries()
            .build()
        return db
    }
}

```

- In our FlowerDAO class we prepared the method for getting all flowers:

- The 'FlowerDAO' file in the project structure and the 'fun getAll(): List<Flower>' method in the code are highlighted with red rectangles."/>

```

7
8 @Dao
9 interface FlowerDAO {
10     @Insert
11     fun insertAll(flower: Flower)
12
13     @Delete
14     fun delete(flower: Flower)
15
16     @Query("SELECT * FROM flower")
17     fun getAll(): List<Flower>
18
19     @Query("SELECT * FROM flower where flowerID IN (:flowerIds)")
20     fun getAllByIds(flowerIds: IntArray): List<Flower>
21 }

```

- Then let's use it, and first let's print out all the flowers we have in the LogCat, then let's add the appropriate implementation in FlowerActivity class:

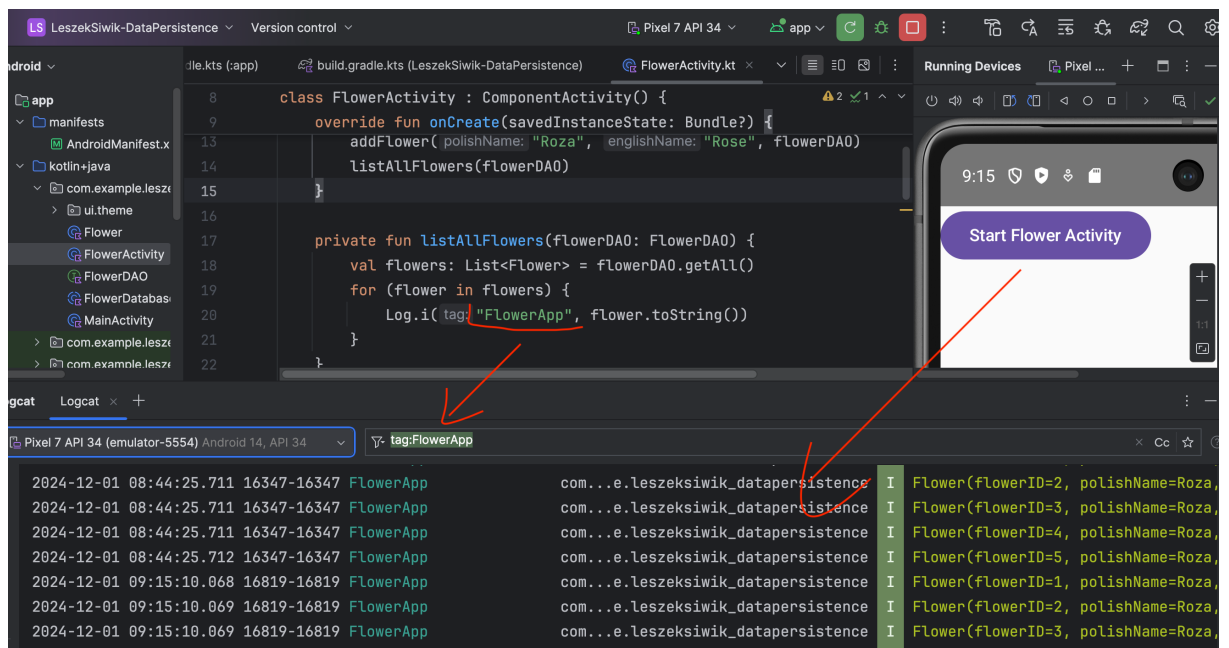
```

class FlowerActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        val db = buildFlowerDatabase()
        val flowerDAO = getFlowerDAO(db)
        addFlower( polishName: "Roza", englishName: "Rose", flowerDAO)
        listAllFlowers(flowerDAO)
    }
}

private fun listAllFlowers(flowerDAO: FlowerDAO) {
    val flowers: List<Flower> = flowerDAO.getAll()
    for (flower in flowers) {
        Log.i( tag: "FlowerApp", flower.toString())
    }
}

```

- And now, when we run the app and start our flower activity we should see the following in the LogCat:



- So now, as for practicing, pls extend the model to keep also the name of the flower in your mother-tongue
- When completed pls take and upload to the UPEL screenshots proving that the task has been completed